Module 6：統計思維與假設檢定

課程定位

所屬課程：2026 Python 數據分析與 AI 基礎：從資料思維到機器學習入門（24 小時）
模組編號：Module 6／共 8 個模組 時長：3 小時 前置模組：M5 視覺化與 EDA 後續模組：M7 ML 前導與學習路線

模組核心主張：沒有統計，你只是在描述現象，不是在判斷現象。描述說「有差異」，統計說「這個差異可靠嗎」。

> "Statistics turns observations into confidence."

學習目標

完成本模組後，學員能夠：

1. 區分母體與樣本、描述統計與推論統計，解釋為什麼光靠均值會誤導決策
2. 說明常態分佈、二項分佈、泊松分佈的直覺意義，並識別哪種業務場景適用哪種分佈
3. 理解信賴區間的含義，正確解讀「95% 信賴區間」的意思（及常見誤解）
4. 解釋假設檢定的邏輯：H0、H1、p-value、顯著水準，並識別常見的誤用陷阱
5. 區分統計顯著性與實際顯著性（effect size），避免「$p < 0.05$ 就代表重要」的誤解
6. 使用 `scipy.stats` 執行單樣本 t 檢定、雙樣本 t 檢定、卡方檢定
7. 設計並解讀一個完整的 A/B Test：從假設設定到樣本數計算到結果解讀
8. 意識到多重比較問題，理解為什麼同時測試 20 個假設會有一個「顯著」的機率

關鍵概念清單

統計基礎

- [ ] 母體（population）vs 樣本（sample）的本質差異
- [ ] 描述統計 vs 推論統計的目的差異
- [ ] 均值、變異數、標準差的直覺意義
- [ ] 常態分佈：為什麼它無所不在？中央極限定理直覺
- [ ] 二項分佈：適用場景（成功/失敗事件）
- [ ] 泊松分佈：適用場景（單位時間內的隨機事件計數）
- [ ] 偏態分佈：收入、房價、用戶停留時間的真實形狀
- [ ] 抽樣分佈與標準誤（Standard Error）

信賴區間與假設檢定

- [ ] 信賴區間：「95% CI」的正確解讀方式
- [ ] 假設檢定邏輯：H0、H1、p-value、顯著水準 α
- [ ] Type I Error（假陽性）vs Type II Error（假陰性）
- [ ] 效果量（Effect Size）：Cohen's d 的直覺
- [ ] 統計顯著（$p < 0.05$）vs 實際顯著（有業務意義）
- [ ] 相關性 vs 因果關係：為什麼觀察研究不能下因果結論

常用統計檢定

- [ ] 單樣本 t 檢定：樣本均值 vs 目標值
- [ ] 雙樣本 t 檢定（獨立樣本）：兩組均值差異
- [ ] 卡方檢定：類別變數的獨立性
- [ ] `scipy.stats` 基本使用：`ttest_1samp`、`ttest_ind`、`chi2_contingency`

A/B Test

- [ ] A/B Test 的實驗設計要素：隨機分組、樣本數計算、測試時間
- [ ] 統計功效（Statistical Power）與樣本數的關係
- [ ] A/B Test 結果解讀：四象限決策矩陣
- [ ] 多重比較問題（Multiple Testing Problem）的直覺理解

投影片大綱

#	投影片標題	核心訊息
1	為什麼分析需要統計？	沒有統計，你只是在描述，不是在判斷
2	母體、樣本與描述 vs 推論統計	統計是不確定性的語言，不是計算器
3	分佈直覺：三種你必須認識的分佈	不同的隨機現象，有不同的自然形狀
4	均值不夠用：為什麼要看分佈？	相同均值，完全不同的故事
5	信賴區間：「我有多確定？」	點估計只給你答案，CI 給你答案的可靠範圍
6	假設檢定：差異可靠嗎？	它問的是：這個差異是不是可靠到足以讓我行動
7	p-value 的正確解讀與常見陷阱	$p < 0.05 \neq$ 差異很大；$\neq$ H0 是假的
8	效果量：統計顯著 vs 實際顯著	百萬樣本讓任何微小差異都「顯著」
9	scipy.stats 實戰：三種常用檢定	用程式碼做出有意義的統計判斷

10	A/B Test 完整案例：從設計到決策	統計是產品決策的底層工具
11	多重比較與相關性陷阱	同時測 20 個，必然有一個「顯著」
12	【練習】業務問題翻譯機 + A/B Test 設計	實作：把業務問題翻成統計語言

詳細投影片內容

Slide 1：為什麼分析需要統計？

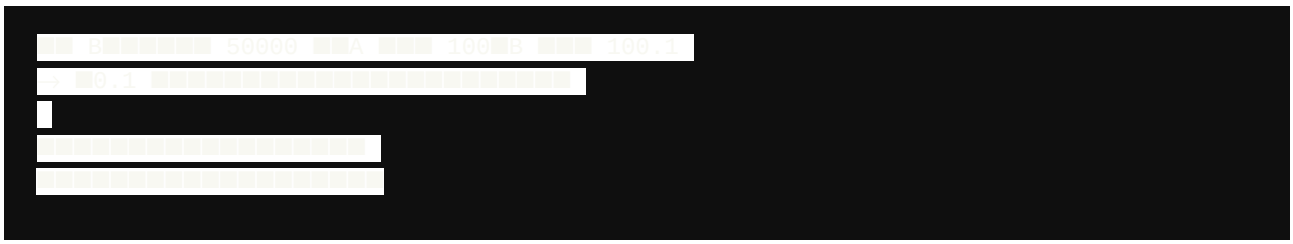
核心訊息：沒有統計，你只是在描述現象，不是在判斷現象。

講師講解要點：

- 情境題：你做了一個新的 App 引導流程，A 版轉換率 12%，B 版轉換率 14%。B 版比較好嗎？
- 純描述的回答：「B 版高了 2 個百分點。」——這是觀察，不是判斷
- 統計的回答：「在樣本數 n=500 的情況下，這 2% 的差異在 95% 信賴水準下無法排除是隨機誤差造成的。」——這是判斷
- 統計的本質：在資料永遠不完整的情況下，用嚴謹的方式量化我們對判斷的信心
- 不用統計的代價：做了很多「看起來有效」但其實只是噪聲的決策，浪費資源，甚至反效果
- 統計不是數學課：不是為了算出答案，而是為了知道答案有多可靠

直覺範例：

```
■■ A■■■■■■■ 50 ■■A ■■■ 100■B ■■■ 105
→ ■5 ■■■■■■■
```



視覺建議： 左右對比：左側「純描述的世界」——一張柱狀圖，A=12%, B=14%，下方標注「看起來 B 贏了」。右側「有統計的世界」——同一張圖加上誤差棒（error bar），誤差棒重疊，下方標注「差異在信心範圍內無法確認」。兩個問題框對比：「哪個高？」vs「差異是真實

銜接下一張：
「要理解統計的判斷力，必須先建立幾個核心概念的直覺：母體、樣本，以及兩種統計的目的。」

Slide 2：母體、樣本與描述 vs 推論統計

核心訊息：統計不是計算器，是不確定性的語言。每個數字背後都有一個關於「我們能知道多少」的假設。

講師講解要點：

- 母體（Population）：你真正想了解的全集——所有用戶、所有訂單、所有未來可能的實驗結果。我們幾
- 樣本（Sample）：我們實際拿到的那一部分。分析的對象是樣本，但判斷的目標是母體
- 母體參數 vs 樣本統計量：
- 母體均值 μ（mu）：未知，是我們想估計的目標
- 樣本均值 $\bar{x}$（x-bar）：已知，是我們手上的估計值
- 標準誤（Standard Error）= $\sigma / \sqrt{n}$：衡量 $\bar{x}$ 有多可能偏離 μ
- 描述統計（Descriptive Statistics）：總結手上的樣本資料——均值、中位數、標準差、分佈形狀。目的：「描述我有什麼」
- 推論統計（Inferential Statistics）：從樣本推斷母體——假設檢定、信賴區間、回歸分析。目的：「推論我不知道的」
- 關鍵直覺：樣本越大，樣本統計量越接近母體參數，不確定性越小

概念對照表：

概念	符號	性質	可得性
母體均值	μ（mu）	固定但未知	幾乎不可得
樣本均值	$\bar{x}$	已知，隨樣本不同而變化	從資料計算
母體標準差	σ（sigma）	固定但未知	幾乎不可得
樣本標準差	s	已知，隨樣本不同而變化	從資料計算
標準誤	$SE = s/\sqrt{n}$	衡量樣本均值的不確定性	從資料計算

程式碼範例：

```
import numpy as np
from scipy import stats

np.random.seed(42)
# ══════════════════════
population = np.random.normal(loc=500, scale=80, size=100_000)
mu_true = population.mean()
print(f"═══════ μ = {mu_true:.2f}")

# ═══════════════
for n in [10, 50, 200, 1000]:
    sample = np.random.choice(population, size=n, replace=False)
    x_bar = sample.mean()
    se = sample.std() / np.sqrt(n)  # ═══
    print(f"  n={n:5d}: x̄={x_bar:.2f}, SE={se:.2f}, ══={abs(x_bar - mu_true):.2f}")

# ═══════════════
sample_means = [np.random.choice(population, size=50).mean() for _ in range(1000)]
print(f"\n1000═══════n=50═══════")
print(f"  ══════ = {np.mean(sample_means):.2f}═══════ μ = {mu_true:.2f}═")
print(f"  ═══════ = {np.std(sample_means):.2f}═══════════")
```

視覺建議：

四個方塊概念圖：母體（大圓虛線）、樣本（小圓實線）、描述統計（總結樣本）、推論統計（從樣本推估母體
越大，SE 越小」的視覺示意：多個小樣本均值形成的分佈，隨 n 增大而收窄。

銜接下一張：

「理解了母體和樣本，我們需要認識幾種常見的資料分佈——因為不同的隨機現象有不同的自然形狀。」

Slide 3：分佈直覺：三種你必須認識的分佈

核心訊息：不同的隨機現象有不同的自然形狀。選對分佈，你的模型才接地氣。

講師講解要點：

- 為什麼要認識分佈？
- 理解資料的「自然形狀」，才能做出正確的統計判斷

假設資料符合常態分佈但實際上是偏態 → 統計結論全錯

常態分佈（Normal Distribution）

- 參數：μ（中心）、σ（寬度）
- 為什麼到處都是？中央極限定理：大量獨立隨機因素疊加的結果，趨向常態
- 現實例子：測量誤差、身高、考試成績

68-95-99.7 法則：μ±1σ 包含 68%，μ±2σ 包含 95%，μ±3σ 包含 99.7%

二項分佈（Binomial Distribution）

- 參數：n（試驗次數）、p（成功機率）
- 適用場景：「n 次獨立試驗中，成功 k 次的機率」
- 現實例子：n 個用戶中有幾個轉換、n 個郵件中有幾個被點開

A/B Test 的轉換率比較就是二項分佈的應用

泊松分佈（Poisson Distribution）

- 參數：λ（單位時間/空間內的平均事件數）
- 適用場景：「固定時間/空間內，罕見事件發生次數」
- 現實例子：每小時客服來電數、每天系統錯誤次數、每週大單數量

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
import seaborn as sns

sns.set_theme(style='whitegrid')
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# ---- 1. [redacted] ----
x = np.linspace(-4, 4, 300)
for mu, sigma, label, color in [(0, 1, 'μ=0, σ=1[redacted]', 'steelblue'),
                                (0, 2, 'μ=0, σ=2[redacted]', 'coral'),
                                (1, 1, 'μ=1, σ=1[redacted]', 'mediumseagreen')]:
    axes[0].plot(x, stats.norm.pdf(x, mu, sigma), label=label, linewidth=2, color=color)
axes[0].axvspan(-1, 1, alpha=0.1, color='steelblue')
axes[0].set_title('[redacted]\n[redacted]')
axes[0].set_xlabel('[redacted]'); axes[0].set_ylabel('[redacted]')
axes[0].legend(fontsize=9)
axes[0].text(0, 0.15, '68%', ha='center', fontsize=10, color='steelblue')

# ---- 2. [redacted] ----
k = np.arange(0, 31)
for n, p, color, label in [(30, 0.3, 'steelblue', 'n=30, p=0.3[redacted]'),
                           (30, 0.5, 'coral',     'n=30, p=0.5[redacted]'),
                           (30, 0.7, 'mediumseagreen', 'n=30, p=0.7[redacted]')]:
    pmf = stats.binom.pmf(k, n, p)
    axes[1].plot(k, pmf, 'o-', label=label, color=color, markersize=4, linewidth=1.5)
axes[1].set_title('[redacted]\n[redacted] k [redacted]')
axes[1].set_xlabel('[redacted] k'); axes[1].set_ylabel('[redacted]')
axes[1].legend(fontsize=9)

# ---- 3. [redacted] ----
k2 = np.arange(0, 20)
for lam, color, label in [(2, 'steelblue', 'λ=2[redacted]'),
                          (5, 'coral',     'λ=5[redacted]'),
                          (10, 'mediumseagreen', 'λ=10[redacted]')]:
    pmf = stats.poisson.pmf(k2, lam)
    axes[2].plot(k2, pmf, 'o-', label=label, color=color, markersize=4, linewidth=1.5)
axes[2].set_title('[redacted]\n[redacted]')
axes[2].set_xlabel('[redacted] k'); axes[2].set_ylabel('[redacted]')
axes[2].legend(fontsize=9)

plt.suptitle('[redacted]', fontsize=14, y=1.02)
plt.tight_layout(); plt.show()

# [redacted]
print("=== [redacted] ===")
print("[redacted]")
pop = np.random.exponential(scale=5, size=100_000)  # [redacted]
fig2, axes2 = plt.subplots(1, 4, figsize=(14, 4))
axes2[0].hist(pop[:500], bins=30, color='coral', alpha=0.7)
axes2[0].set_title('[redacted]')
for i, n in enumerate([5, 30, 100], 1):
    means = [np.random.choice(pop, n).mean() for _ in range(2000)]
    axes2[i].hist(means, bins=30, color='steelblue', alpha=0.7)
    axes2[i].set_title(f'[redacted] n={n}[redacted]')
plt.suptitle('[redacted]n [redacted]', y=1.02)
plt.tight_layout(); plt.show()
```

視覺建議：
三列對照，每列一種分佈：分佈名稱、數學直覺（用文字，不用公式）、現實業務場景例子、圖形示意。底部加
→ 是固定試驗次數的成功計數嗎 → 二項；是單位時間內的罕見事件 → 泊松；是連續數值且中央極限定理適用 → 常態。

銜接下一張：「理解了分佈，我們回到一個核心問題：為什麼光看均值不夠？」

Slide 4：均值不夠用：為什麼要看分佈？

核心訊息：均值是一個點，分佈是完整的故事。兩組相同均值的資料，可以有完全不同的業務含義。

講師講解要點：

- 均值的陷阱：比爾・蓋茲走進一間酒吧，酒吧裡所有人的「平均財富」立刻暴增——但沒有人變富了。均
- 為什麼看分佈更重要：
- 分佈讓你看到「大多數人在哪裡」而不只是「中心在哪裡」
- 雙峰分佈意味著存在兩個完全不同的用戶群體，用均值代表他們是荒唐的
- 右偏分佈（收入、房價）中，均值 >> 中位數，用均值描述「典型用戶」會嚴重誤導
- 偏度（Skewness）的直覺：
- 正偏（右偏）：長尾在右，均值 > 中位數。收入、消費金額、留存天數
- 負偏（左偏）：長尾在左，均值 < 中位數。考試成績（難度低時）
- 多種集中趨勢量數的對比：
- 均值：受極端值影響最大，代表「數學重心」
- 中位數：不受極端值影響，代表「典型值」
- 眾數：出現最多次的值，代表「最常見值」

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style='whitegrid')
```

```
np.random.seed(42)
[redacted]
fig, axes = plt.subplots(2, 2, figsize=(13, 10))
[redacted]
# ---- [redacted] Anscombe [redacted] ----
np.random.seed(42)
group_a = np.random.normal(500, 50, 500)                    # [redacted]
group_b = np.concatenate([np.random.normal(300, 30, 250),   # [redacted]
                          np.random.normal(700, 30, 250)])
# [redacted]
group_b = group_b - group_b.mean() + group_a.mean()
[redacted]
axes[0,0].hist(group_a, bins=40, alpha=0.6, label=f'A [redacted]={group_a.mean():.0f}[redacted]', color='steelb
axes[0,0].hist(group_b, bins=40, alpha=0.6, label=f'B [redacted]={group_b.mean():.0f}[redacted]', color='coral'
axes[0,0].axvline(group_a.mean(), color='navy',   linestyle='--', linewidth=2)
axes[0,0].axvline(group_b.mean(), color='darkred', linestyle='--', linewidth=2)
axes[0,0].set_title('[redacted]\n[redacted] ≠ [redacted]')
axes[0,0].legend()
[redacted]
# ---- [redacted]2[redacted] vs [redacted] vs [redacted] ----
income = np.concatenate([np.random.lognormal(10, 0.5, 950),
                         np.random.uniform(5e6, 1e7, 50)])  # [redacted]
mean_inc = income.mean()
median_inc = np.median(income)
mode_approx = stats.mode(income.astype(int))[0]
[redacted]
axes[0,1].hist(income, bins=100, color='steelblue', alpha=0.7)
axes[0,1].axvline(mean_inc,   color='red',   linestyle='--', linewidth=2, label=f'[redacted] {mean_inc/1
axes[0,1].axvline(median_inc, color='green', linestyle='--', linewidth=2, label=f'[redacted] {median_in
axes[0,1].set_xlim(0, income.quantile(0.99) if hasattr(income, 'quantile') else np.percentile(inco
axes[0,1].set_title('[redacted]\n[redacted]')
axes[0,1].legend()
axes[0,1].set_xlabel('[redacted]')
[redacted]
# [redacted] xlim
axes[0,1].set_xlim(0, np.percentile(income, 99))
[redacted]
# ---- [redacted]3[redacted] ----
x = np.linspace(0, 1000, 300)
for sigma, color, label in [(50, 'steelblue', 'σ=50[redacted]'),
                            (150, 'coral', 'σ=150[redacted]'),
                            (300, 'mediumseagreen', 'σ=300[redacted]')]:
    from scipy.stats import norm
    axes[1,0].plot(x, norm.pdf(x, 500, sigma), label=label, linewidth=2, color=color)
axes[1,0].axvline(500, color='black', linestyle=':', linewidth=1.5, label='[redacted]=500')
axes[1,0].set_title('[redacted]\n[redacted]')
axes[1,0].legend(); axes[1,0].set_xlabel('[redacted]')
[redacted]
# ---- [redacted]4[redacted] ----
right_skew = np.random.lognormal(3, 1, 1000)
normal     = np.random.normal(right_skew.mean(), right_skew.std()*0.5, 1000)
axes[1,1].hist(right_skew, bins=50, alpha=0.6, label=f'[redacted]={stats.skew(right_skew):.1f}[redacted]', col
axes[1,1].hist(normal,     bins=50, alpha=0.6, label=f'[redacted]={stats.skew(normal):.1f}[redacted]', color=
axes[1,1].axvline(right_skew.mean(), color='darkred', linestyle='--', linewidth=1.5)
axes[1,1].axvline(np.median(right_skew), color='red', linestyle=':', linewidth=1.5)
```

```
axes[1,1].set_title('[redacted]\n[redacted]')
axes[1,1].legend()

plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：重點標注第一個子圖（相同均值的雙峰 vs 單峰），用紅色虛線指出兩條均值線重疊，但分佈形狀截然不同。旁邊加文字框：「如果你只回報均值，你的老

銜接下一張：
「知道了分佈，我們現在需要一個工具，讓估計值帶著不確定性一起傳遞——這就是信賴區間。」

Slide 5：信賴區間：「我有多確定？」

核心訊息：點估計只給你一個答案，信賴區間給你「答案有多可靠」的範圍。

講師講解要點：

- 為什麼需要信賴區間？
- 樣本均值 $\bar{x}$ 只是一個點估計。每次重新抽樣都會得到不同的 $\bar{x}$
- 信賴區間告訴我們：「如果重複做 100 次這個研究，約有 95 次，真實的母體均值 μ 會落在這個區間內」

它量化了我們的不確定性，讓決策更有根據

95% CI 的正確解讀：

- ☒ 正確：「在相同方法重複抽樣 100 次下，約有 95 個區間會包含真實母體均值」
- ☒ 錯誤：「母體均值有 95% 的機率落在這個區間」（母體均值是固定的，不是隨機的！）

☒ 錯誤：「95% 的資料落在這個區間」（這是說分佈範圍，不是說 CI）

CI 越窄越好嗎？

- 更窄的 CI = 更高的確定性，但需要更大的樣本數
- 寬 CI + 不包含零 → 有差異但不精確
- 窄 CI + 不包含零 → 有差異且很精確（最好的情況）

任何 CI 包含零 → 無法排除「沒有差異」

CI 的計算直覺：CI = x̄ ± z × SE，其中 SE = s / √n

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ---- [redacted] ----
population_mean = 500  # [redacted]
population = np.random.normal(population_mean, 80, 100_000)

n = 50
num_experiments = 100
contained = 0

fig, ax = plt.subplots(figsize=(10, 8))
for i in range(num_experiments):
    sample = np.random.choice(population, size=n)
    x_bar = sample.mean()
    se = sample.std() / np.sqrt(n)
    ci_low = x_bar - 1.96 * se
    ci_high = x_bar + 1.96 * se

    color = 'steelblue' if ci_low <= population_mean <= ci_high else 'red'
    if color == 'steelblue':
        contained += 1
    ax.plot([ci_low, ci_high], [i, i], color=color, alpha=0.6, linewidth=1.5)
    ax.plot(x_bar, i, 'o', color=color, markersize=3)

ax.axvline(population_mean, color='black', linestyle='--', linewidth=2, label=f'[redacted] μ={popula
ax.set_xlabel('[redacted]')
ax.set_title(f'100 [redacted] 95% [redacted]\n[redacted]={contained} [redacted] μ[redacted]={100-contained} [redacted]\n[redacted] 95
ax.legend(); plt.tight_layout(); plt.show()

# ---- [redacted] CI [redacted] ----
sample_from_pop = np.random.choice(population, size=1000)  # [redacted]
print('--- [redacted] vs [redacted] ---')
for n_size in [10, 30, 100, 500, 1000]:
    sample_n = sample_from_pop[:n_size]
    ci = stats.t.interval(0.95, df=n_size-1,
                          loc=sample_n.mean(),
                          scale=stats.sem(sample_n))
    width = ci[1] - ci[0]
    print(f'  n={n_size:4d}: CI=[{ci[0]:.1f}, {ci[1]:.1f}], [redacted]={width:.1f}, [redacted]={sample_n.mean()

# ---- [redacted] CI ----
sample = np.random.normal(520, 75, 80)
ci_95 = stats.t.interval(0.95, df=len(sample)-1, loc=sample.mean(), scale=stats.sem(sample))
```

```
ci_99 = stats.t.interval(0.99, df=len(sample)-1, loc=sample.mean(), scale=stats.sem(sample))
print(f"\n[redacted]n={len(sample)}[redacted]={sample.mean():.1f}, SE={stats.sem(sample):.1f}")
print(f"95% CI: [{ci_95[0]:.1f}, {ci_95[1]:.1f}] [redacted]={ci_95[1]-ci_95[0]:.1f}")
print(f"99% CI: [{ci_99[0]:.1f}, {ci_99[1]:.1f}] [redacted]={ci_99[1]-ci_99[0]:.1f}")
```

視覺建議：重點展示 100 個 CI 的圖，大多數藍色（包含真實值），少數紅色（未包含），直覺理解「95% CI 的含義」。旁邊加一個「CI 寬度 vs 樣本數」的反比關係圖。

銜接下一張：
「信賴區間給了我們估計的可靠範圍。假設檢定更進一步——它讓我們用嚴謹的邏輯做出是/否的判斷。」

Slide 6：假設檢定：差異可靠嗎？

核心訊息：假設檢定問的不是「有沒有差異」，而是「這個差異可靠到足以讓我行動嗎」。

講師講解要點：

- 假設檢定的核心邏輯（法庭類比）：
- H0（虛無假設）= 無罪推定：預設「沒有差異，你看到的差異只是抽樣噪聲」
- H1（對立假設）= 起訴理由：「有差異，而且這個差異是真實的」
- p-value = 反對無罪的證據強度：「如果 H0 是真的，看到這麼大差異的機率是多少？」
- 顯著水準 α = 定罪門檻：通常設為 0.05

結論：$p < \alpha \rightarrow$ 拒絕 H0（定罪）；$p \geq \alpha \rightarrow$ 無法拒絕 H0（無罪釋放）

假設檢定的五步驟：

- 設定 H0 和 H1（H0 永遠是「沒有效果」的假設）
- 選擇顯著水準 α（最常用 0.05，醫療或飛安用 0.01）
- 收集資料，計算檢定統計量（t 值、z 值、χ^2 值）
- 計算 p-value

根據 p vs α 做決定

Type I Error vs Type II Error：

- Type I（α 錯誤，假陽性）：H0 是真的，但我們拒絕了它。「冤枉好人」
- Type II（β 錯誤，假陰性）：H0 是假的，但我們沒有拒絕它。「放跑壞人」
- 統計功效（Power）= 1 - β：正確拒絕假 H0 的機率，目標通常 ≥ 0.8

假設設定練習：

```
[illegible]
|
H0: [illegible] ≤ [illegible]
    [illegible] μ_new ≤ μ_old [illegible] ≤ 0
|
H1: [illegible] > [illegible]
    [illegible] μ_new > μ_old [illegible] > 0
|
[illegible] α = 0.05
[illegible] p < 0.05 [illegible]
```

視覺建議： 流程圖：情境描述 → 設定 H0/H1 → 收集資料 → 計算統計量 → 計算 p-value → 與 α 比較 → 做決定。旁邊加「法庭類比」對照表，以及 Type I/II Error 的 2x2 矩陣（真實情況 × 我們的決定）。

銜接下一張： 「p-value 是假設檢定的核心輸出，但它也是最常被誤解的統計概念之一。讓我們把誤解一個個拆除。」

Slide 7：p-value 的正確解讀與常見陷阱

核心訊息：p < 0.05 不代表差異很大，不代表 H0 是假的，也不代表結果可以重現。

講師講解要點：

p-value 的正確定義：「如果 H0 為真（真的沒有差異），那麼我們觀察到這麼大（或更大）差異的機率。」 p = 0.03 意思是：「如果根本沒有差異，我有 3% 的機率看到這麼大的差異。」

常見誤解逐一破解：

誤解	正確理解

$p < 0.05$ 代表差異很大	不是。p 只說差異是否可靠，不說差異有多大
$p = 0.03$ 代表 H0 有 3% 機率是真的	不是。p 是在 H0 為真的條件下計算的，不是 H0 為真的機率
$p > 0.05$ 代表 H0 是真的	不是。只是證據不足以拒絕 H0，而不是證明 H0
$p < 0.05$ 代表結果一定能重現	不是。樣本波動仍然存在
$\alpha = 0.05$ 是自然法則	不是。這是一個慣例，應根據決策代價調整

- 單尾 vs 雙尾檢定：
- 雙尾（two-tailed）：問「有沒有差異」（方向未知）
- 單尾（one-tailed）：問「是不是比較大/小」（方向已知）
- 建議：除非有強力理論依據，預設用雙尾檢定，更保守

視覺建議：左側顯示常態分佈，標注臨界區域（拒絕 H0 的區域）和 p-value 的位置。右側顯示「誤解清單」，每個誤解用紅色 ☒ 標記，正確理解用綠色 ✓ 標記。底部加一個「p-value 單頁速查卡」。

銜接下一張：「p-value 只說差異是否可靠，但不說差異有多大。這個「多大」的問題，需要效果量來回答。」

Slide 8：效果量：統計顯著 vs 實際顯著

核心訊息：百萬樣本能讓任何微小差異都「統計顯著」——但 0.001% 的轉換率提升值得全面改版嗎？

講師講解要點：

- 為什麼需要效果量（Effect Size）？
- 統計顯著性受樣本數影響極大：n 足夠大，任何差異都會 $p < 0.05$
- 效果量衡量差異的「實際大小」，獨立於樣本數

決策應該同時考慮：統計顯著（差異是真實的嗎？）AND 實際顯著（差異夠大嗎？）

Cohen's d（連續變數效果量）：

- d = (μ[redacted] - μ[redacted]) / s_pooled
- 解讀：d = 0.2（小效果），d = 0.5（中等效果），d = 0.8（大效果）

直覺：「兩組均值差了幾個標準差？」

業務決策四象限：

統計顯著（p < 0.05）	統計不顯著（p ≥ 0.05）
實際顯著（有大效果量）：支持大改善	延長測試，或增加樣本數
實際不顯著（微效果）：評估 ROI	維持現狀，沒有證據也沒有意義

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ═══ [redacted] ═══
print("═══ [redacted] p-value [redacted] ═══")
true_diff = 0.5   # [redacted] 0.5 [redacted]
std       = 50.0  # [redacted] 50 [redacted]

for n in [30, 100, 500, 2000, 10000, 100000]:
    # [redacted]
    group_a = np.random.normal(500, std, n)
    group_b = np.random.normal(500 + true_diff, std, n)
    t_stat, p_val = stats.ttest_ind(group_a, group_b)
    cohens_d = true_diff / std  # [redacted]
    print(f"  n={n:7d}: p={p_val:.4f} {'***' [redacted]' if p_val < 0.05 else '    [redacted]'},"
          f"Cohen's d={cohens_d:.3f}[redacted]{'[redacted]' if cohens_d > 0.8 else '[redacted]' if cohens_d > 0.5 else '[redacted]' i

# ═══ [redacted] p-value ═══
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sample_sizes = [30, 50, 100, 200, 500, 1000, 5000, 10000]
```

```
p_values = []
for n in sample_sizes:
    a = np.random.normal(500, 50, n)
    b = np.random.normal(501, 50, n)  # 微小差異 d ≈ 0.02
    _, p = stats.ttest_ind(a, b)
    p_values.append(p)

axes[0].plot(sample_sizes, p_values, 'o-', color='steelblue', linewidth=2)
axes[0].axhline(0.05, color='red', linestyle='--', linewidth=1.5, label='α = 0.05')
axes[0].set_xscale('log')
axes[0].set_xlabel('樣本數 (log scale)'); axes[0].set_ylabel('p-value')
axes[0].set_title('效果量固定 d=0.02，樣本越大越顯著\n(效果量不變，只是統計力量增加)')
axes[0].legend()

# Cohen's d 直覺
x = np.linspace(-4, 6, 300)
for d, label, color in [(0.2, "小效果 (d=0.2)", 'steelblue'),
                        (0.5, "中效果 (d=0.5)", 'coral'),
                        (0.8, "大效果 (d=0.8)", 'mediumseagreen')]:
    axes[1].plot(x, stats.norm.pdf(x, 0, 1), color=color, linewidth=2, linestyle='--', alpha=0.5)
    axes[1].plot(x, stats.norm.pdf(x, d, 1), color=color, linewidth=2, label=label)
axes[1].axvline(0, color='black', linestyle=':', linewidth=1)
axes[1].set_title("Cohen's d 直覺：兩組分佈重疊程度")
axes[1].set_xlabel('標準差'); axes[1].legend()

plt.suptitle('樣本數 vs 效果量', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議： 左圖展示「樣本數對 p-value 的影響」（效果量固定但樣本越大越顯著），右圖展示 Cohen's d 的直覺（兩組分佈重疊程度）。底部放決策四象限表格。

銜接下一張：「理論夠了，來看程式碼怎麼做——scipy.stats 讓統計判斷變成三行 Python。」

Slide 9：scipy.stats 實戰：三種常用檢定

核心訊息：用程式碼落地統計判斷，掌握 scipy.stats 的三個核心 API。

講師講解要點：

- 單樣本 t 檢定：我的樣本均值和目標值（或歷史值）有沒有顯著差異？
- 使用：`stats.ttest_1samp(sample, popmean=target)`
- 獨立雙樣本 t 檢定：兩組樣本的均值有沒有顯著差異？
- 使用：`stats.ttest_ind(group_a, group_b, equal_var=False)`
- 注意：`equal_var=False` 使用 Welch's t-test，不假設兩組變異數相等（更保守，更常用）

- 卡方獨立性檢定：兩個類別變數之間有沒有關聯？
- 使用：`stats.chi2_contingency(contingency_table)`
- 適合：點擊率 vs 設備類型、轉換率 vs 來源渠道

完整程式碼範例：

```
import numpy as np
from scipy import stats
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
print("="*60)
print("[redacted] t [redacted] NPS [redacted] 70 [redacted]")
print("="*60)
nps_scores = np.random.normal(72, 15, 80)  # [redacted] NPS
target_nps = 70
t_stat, p_val = stats.ttest_1samp(nps_scores, popmean=target_nps)
print(f"[redacted]: {nps_scores.mean():.2f}")
print(f"t [redacted]: {t_stat:.3f}")
print(f"p-value: {p_val:.4f}")
if p_val < 0.05:
    direction = "[redacted]" if nps_scores.mean() > target_nps else "[redacted]"
    print(f"[redacted]: [redacted] → [redacted] NPS [redacted]{direction}[redacted] {target_nps} [redacted]95% [redacted]")
else:
    print(f"[redacted]: [redacted] → [redacted] NPS [redacted] {target_nps} [redacted]")

print("\n" + "="*60)
print("[redacted] t [redacted] vs [redacted]")
print("="*60)
desktop_orders = np.random.normal(650, 120, 150)
mobile_orders = np.random.normal(580, 160, 300)  # [redacted]
t_stat, p_val = stats.ttest_ind(desktop_orders, mobile_orders, equal_var=False)
cohens_d = (desktop_orders.mean() - mobile_orders.mean()) / \
           np.sqrt((desktop_orders.std()**2 + mobile_orders.std()**2) / 2)
print(f"[redacted]: {desktop_orders.mean():.2f}[redacted]n={len(desktop_orders)}[redacted]")
print(f"[redacted]: {mobile_orders.mean():.2f}[redacted]n={len(mobile_orders)}[redacted]")
print(f"[redacted]: {desktop_orders.mean() - mobile_orders.mean():.2f}")
print(f"t [redacted]: {t_stat:.3f}  p-value: {p_val:.4f}")
print(f"Cohen's d: {cohens_d:.3f} ({'[redacted]' if abs(cohens_d) > 0.8 else '[redacted]' if abs(cohens_d) > 0.5 el
if p_val < 0.05:
    print("[redacted]: [redacted]")
    print(f"  [redacted] {desktop_orders.mean()-mobile_orders.mean():.0f} [redacted]
          f"[redacted] {abs(cohens_d):.2f}[redacted]{'[redacted]' if abs(cohens_d) > 0.5 else '[redacted]'}[redacted]")
else:
    print("[redacted]: [redacted]")

print("\n" + "="*60)
print("[redacted]")
print("="*60)
```

```
[redacted]vs [redacted]
contingency = pd.DataFrame({
    '[redacted]': [850, 720, 680, 195],
    '[redacted]': [150, 280, 120, 105]
}, index=['organic', 'paid', 'social', 'referral'])
print("[redacted]")
print(contingency)
print(f"\n[redacted]")
print((contingency['[redacted]'] / contingency.sum(axis=1)).round(3))
[redacted]
chi2, p_val, dof, expected = stats.chi2_contingency(contingency)
print(f"\n[redacted]: {chi2:.3f}, df={dof}")
print(f"p-value: {p_val:.6f}")
if p_val < 0.05:
    print("[redacted]: [redacted]")
else:
    print("[redacted]: [redacted]")
```

視覺建議：
三段程式碼用不同色彩框分隔，每段後面標注「適用情境」。右側加一個「選擇哪種檢定」決策樹：比較均值 → 一組 vs 目標值 → 單樣本 t；兩組均值 → 雙樣本 t；類別 vs 類別 → 卡方。

銜接下一張：「現在把所有概念整合在一個完整的 A/B Test 案例裡，從設計到決策走一遍。」

Slide 10：A/B Test 完整案例：從設計到決策

核心訊息：A/B Test 是用統計把產品決策從「感覺上比較好」變成「數據上可以確認更好」。

業務情境：[redacted]電商平台在考慮改版結帳頁面（簡化步驟），預期可提升轉換率。目前轉換率：18%。希望能提升至 21%（效果量 +3 個百分點）。

完整 A/B Test 流程：

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
[redacted]
sns.set_theme(style='whitegrid')
np.random.seed(42)
[redacted]
[redacted]
print("=" * 60)
print("[redacted]")
print("=" * 60)
```

[redacted]

```
p_control = 0.18  # [redacted]
p_treatment = 0.21  # [redacted]
alpha = 0.05  # [redacted]
power = 0.80  # [redacted]80% [redacted]

# [redacted]using [redacted]
def calc_sample_size(p1, p2, alpha=0.05, power=0.80):
    z_alpha2 = stats.norm.ppf(1 - alpha/2)  # [redacted]
    z_beta = stats.norm.ppf(power)
    p_bar = (p1 + p2) / 2
    n = ((z_alpha2 * np.sqrt(2 * p_bar * (1 - p_bar)) +
         z_beta * np.sqrt(p1*(1-p1) + p2*(1-p2))) / (p2 - p1)) ** 2
    return int(np.ceil(n))

n_required = calc_sample_size(p_control, p_treatment, alpha, power)
print(f"[redacted]: {p_control:.0%}")
print(f"[redacted]: {p_treatment:.0%}[redacted] MDE = {p_treatment-p_control:.0%}[redacted]")
print(f"[redacted] α: {alpha}")
print(f"[redacted]: {power:.0%}")
print(f"[redacted]: {n_required}")
print(f"[redacted]: {n_required * 2}[redacted] {n_required}[redacted]")
print(f"[redacted] 5000[redacted] {n_required * 2 / 5000:.1f} [redacted]")

# ============ [redacted] 2[redacted]============
print("\n" + "=" * 60)
print("[redacted] 2[redacted]")
print("=" * 60)
n_actual = n_required  # [redacted]
control = np.random.binomial(1, p_control, n_actual)
treatment = np.random.binomial(1, p_treatment + 0.005, n_actual)  # [redacted]

conv_c = control.mean()
conv_t = treatment.mean()
print(f"[redacted]A[redacted]: n={n_actual}, [redacted]={conv_c:.2%}")
print(f"[redacted]B[redacted]: n={n_actual}, [redacted]={conv_t:.2%}")
print(f"[redacted]: {conv_t - conv_c:+.2%}")

# ========== [redacted] 3[redacted] ==========
print("\n" + "=" * 60)
print("[redacted] 3[redacted]")
print("=" * 60)
print("H0[redacted] ≤ [redacted]")
print("H1[redacted] > [redacted]")

# [redacted] z [redacted]
from statsmodels.stats.proportion import proportions_ztest
count = np.array([treatment.sum(), control.sum()])
nobs = np.array([n_actual, n_actual])
z_stat, p_val = proportions_ztest(count, nobs, alternative='larger')

print(f"\nz [redacted]: {z_stat:.3f}")
print(f"p-value: {p_val:.4f}")

# [redacted]
```

```
effect_size = conv_t - conv_c
relative_lift = (conv_t - conv_c) / conv_c
print(f"[redacted]: {effect_size:+.2%}")
print(f"[redacted]: {relative_lift:+.1%}")
[redacted]
# ========== [redacted] 4[redacted] ==========
print("\n" + "=" * 60)
print("[redacted] 4[redacted]95% [redacted]")
print("=" * 60)
from statsmodels.stats.proportion import confint_proportions_2indep
ci = confint_proportions_2indep(treatment.sum(), n_actual,
                                control.sum(),   n_actual,
                                method='agresti-caffo')
print(f"[redacted] 95% CI: [{ci[0]:.3%}, {ci[1]:.2%}]")
if ci[0] > 0:
    print("CI [redacted] > 0[redacted]")
else:
    print("CI [redacted]")
[redacted]
# ========== [redacted] 5[redacted] ==========
print("\n" + "=" * 60)
print("[redacted] 5[redacted]")
print("=" * 60)
[redacted]
stat_sig = p_val < alpha
practical_sig = abs(effect_size) >= 0.02  # [redacted] 2 [redacted]
[redacted]
if stat_sig and practical_sig:
    decision = "[redacted]"
    reason = f"[redacted]p={p_val:.4f}<{alpha}[redacted]{effect_size:+.2%} > 2%[redacted]"
elif stat_sig and not practical_sig:
    decision = "[redacted] ROI [redacted]"
    reason = f"[redacted]{effect_size:+.2%} < 2%[redacted]"
else:
    decision = "[redacted]"
    reason = f"[redacted]p={p_val:.4f}[redacted]"
[redacted]
print(f"[redacted]{decision}")
print(f"[redacted]{reason}")
[redacted]
# ========== [redacted] ==========
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
[redacted]
# [redacted]
bars = axes[0].bar(["[redacted]", "[redacted]"], [conv_c, conv_t],
                   color=['#3498db', '#2ecc71'], alpha=0.85, width=0.5)
axes[0].axhline(conv_c, color='blue', linestyle='--', alpha=0.5)
for bar, val in zip(bars, [conv_c, conv_t]):
    axes[0].text(bar.get_x() + bar.get_width()/2, val + 0.005, f"{val:.2%}",
                 ha='center', fontsize=12, fontweight='bold')
axes[0].set_title(f"A/B Test [redacted]\np={p_val:.4f}[redacted]={effect_size:+.2%}[redacted]{"[redacted]" if stat_sig else "[redacted]
axes[0].set_ylabel("[redacted]"); axes[0].set_ylim(0, max(conv_c, conv_t) * 1.3)
```

```
[redacted]
[redacted]
matrix_data = [['[redacted]\n[redacted]', '[redacted]\n[redacted]'],
               ['[redacted] ROI\n[redacted]', '[redacted]\n[redacted]']]
colors_matrix = [['#2ecc71', '#f39c12'], ['#3498db', '#95a5a6']]
[redacted]
axes[1].axis('off')
table = axes[1].table(cellText=matrix_data,
                      rowLabels=['[redacted]\n([redacted])', '[redacted]\n([redacted])'],
                      colLabels=['[redacted]\n(p < 0.05)', '[redacted]\n(p ≥ 0.05)'],
                      cellColours=colors_matrix,
                      loc='center', cellLoc='center')
table.scale(1.5, 3)
table.auto_set_font_size(False); table.set_fontsize(10)
axes[1].set_title('A/B Test [redacted]', fontsize=12, pad=20)
[redacted]
[redacted]
current_row = 0 if practical_sig else 1
current_col = 0 if stat_sig else 1
print(f"\n[redacted] {current_row+1} [redacted] {current_col+1} [redacted]")
[redacted]
plt.suptitle('A/B Test [redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：上方顯示 A/B Test 全流程（五個步驟的標籤框），下方左側是轉換率柱狀圖（含誤差棒），右側是決策四象限矩陣，用顏色標注當

銜接下一張：「A/B Test 是強大的工具，但有兩個常見陷阱必須認識：多重比較問題和相關性的因果誤解。」

Slide 11：多重比較與相關性陷阱

核心訊息：同時測試 20 個假設，必然有一個「顯著」——這不是統計的成功，是多重比較的失敗。

講師講解要點：

多重比較問題（Multiple Testing Problem）：

- 單次檢定，α = 0.05：假陽性率 = 5%
- 同時做 20 個獨立檢定：至少一個假陽性的機率 = 1 - (0.95)^20 ≈ 64%
- 業務現場的常見陷阱：同時測試「點擊率、轉換率、客均價、停留時間、跳出率...」20 個指標
- 解決方案：Bonferroni 校正（最保守）：α' = α / m，其中 m 是檢定次數

- 做 20 個檢定時，每個的顯著水準降為 0.05 / 20 = 0.0025
- 更實際的做法：事前聲明主要指標（pre-registration），只有主要指標需要嚴格校正

相關性與因果關係的界限：

- 觀察到 X 和 Y 相關，可能的原因：
- X 導致 Y（X → Y）
- Y 導致 X（反向因果）
- 有第三變數 Z 同時導致 X 和 Y（混淆變數）
- 純屬巧合（偽相關）
- 隨機對照實驗（RCT）才能建立因果關係，因為隨機分組消除了混淆變數
- A/B Test 是在網際網路公司最接近 RCT 的方法

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ---- [redacted] ----
print("=== [redacted] 10000 [redacted] ===")
print("[redacted]A/B [redacted]H0 [redacted]")

num_simulations = 10000
num_tests       = 20  # [redacted] 20 [redacted]
false_positive_any = 0

for _ in range(num_simulations):
    found_any_significant = False
    for _ in range(num_tests):
        a = np.random.normal(0, 1, 50)
        b = np.random.normal(0, 1, 50)  # [redacted]
        _, p = stats.ttest_ind(a, b)
        if p < 0.05:
            found_any_significant = True
            break
    if found_any_significant:
        false_positive_any += 1

print(f"\n[redacted] {num_simulations} [redacted]")
```

```
print(f"  [redacted]{false_positive_any/num_simulations:.1%}[redacted] 64%[redacted]")
print(f"  [redacted]1 - (0.95)^20 = {1-(0.95)**20:.1%}")
[redacted]
print("\n=== Bonferroni [redacted] ===")
alpha_adjusted = 0.05 / num_tests
print(f"[redacted] α: 0.05")
print(f"[redacted] α: {alpha_adjusted:.4f}[redacted] = 0.05 / {num_tests}[redacted]")
print(f"[redacted]Type II Error [redacted]")
[redacted]
# ═══ [redacted] ═══
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
[redacted]
num_tests_range = range(1, 51)
fp_probs = [1 - (1 - 0.05)**k for k in num_tests_range]
axes[0].plot(num_tests_range, fp_probs, color='coral', linewidth=2)
axes[0].axhline(0.05, color='steelblue', linestyle='--', linewidth=1.5, label='[redacted] 5%')
axes[0].axhline(0.64, color='red', linestyle=':', linewidth=1.5, label='20 [redacted] 64%')
axes[0].fill_between(num_tests_range, fp_probs, 0.05, alpha=0.2, color='coral')
axes[0].set_xlabel('[redacted]')
axes[0].set_ylabel('[redacted]')
axes[0].set_title('[redacted]\n[redacted]')
axes[0].legend(), axes[0].set_ylim(0, 1)
[redacted]
# [redacted]Spurious Correlation[redacted]
years = np.arange(1999, 2010)
# [redacted]
pool_drownings = [109, 102, 102, 98, 85, 95, 96, 98, 123, 94, 102][:len(years)]
cage_movies    = [2, 2, 2, 3, 1, 1, 2, 3, 4, 1, 4][:len(years)]
[redacted]
ax2 = axes[1]
color_a, color_b = 'steelblue', 'coral'
ln1 = ax2.plot(years, pool_drownings, 'o-', color=color_a, linewidth=2, label='[redacted]')
ax2.set_xlabel('[redacted]'); ax2.set_ylabel('[redacted]', color=color_a)
ax2_twin = ax2.twinx()
ln2 = ax2_twin.plot(years, cage_movies, 's--', color=color_b, linewidth=2, label='Nicolas Cage [redacted]
ax2_twin.set_ylabel('Nicolas Cage [redacted]', color=color_b)
[redacted]
r, p = stats.pearsonr(pool_drownings[:len(years)], cage_movies[:len(years)])
ax2.set_title(f'[redacted]\nr = {r:.2f}[redacted]p = {p:.3f}\n[redacted]')
[redacted]
lns = ln1 + ln2
labels = [l.get_label() for l in lns]
ax2.legend(lns, labels, loc='upper left')
[redacted]
plt.suptitle('[redacted] + [redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
[redacted]
# ═══ [redacted] ═══
print("\n=== [redacted] ===")
causal_criteria = {
    "[redacted]": "X [redacted] Y [redacted]",
    "[redacted]": "X [redacted] Y [redacted]",
    "[redacted]": "[redacted] Z [redacted] X [redacted] Y[redacted]",
    "[redacted]": "[redacted] X [redacted] Y[redacted]",
    "[redacted]": "[redacted]X [redacted]A/B test[redacted]",
```

```
for k, v in causal_criteria.items():
    print(f"  [{k}] {v}")
```

視覺建議：左圖展示「檢定次數 vs 假陽性率」的曲線（隨 m 增大快速上升），右圖展示一個著名的偽相關例子（Nicolas Cage 電影數 vs 泳池溺水人數），用幽默的方式強化「相關 ≠ 因果」的印象。

銜接下一張：「把所有概念整合在一起，輪到你把業務問題翻譯成統計問題了。」

Slide 12：【練習】業務問題翻譯機 + A/B Test 設計

核心訊息：統計的實用價值在於，它能讓你把「我覺得」換成「資料顯示，在 X% 信心水準下」。

練習 A：業務問題翻譯機（20 分鐘）

將以下三個業務問題翻譯成可被統計回答的統計問題：

#	業務問題	翻譯目標
1	「新的 App 推播設計讓用戶更願意打開嗎？」	翻譯成 A/B Test 假設檢定設計
2	「週末的平均訂單金額真的比平日高嗎？」	翻譯成雙樣本 t 檢定設計
3	「我們的用戶留存率有沒有在最近三個月改善？」	翻譯成時序比較 + 信賴區間設計

翻譯格式模板：

```
[0] [redacted]
[1] [redacted]
[redacted]
[redacted]
[redacted] ___ [redacted] / [redacted] / [redacted] z [redacted] / ...
```

[redacted]

練習 B：A/B Test 設計（15 分鐘）

情境：你的電商平台首頁目前有一個「通知欄」，你的 PM 想把它改成「個人化推薦欄」。你需要設計一個 A/B Test 來評估這個改版。

設計要求：

1. 定義主要指標（Primary Metric）和次要指標（Secondary Metrics）
2. 設定 H0 和 H1
3. 計算最小需要的樣本數（假設：目前點擊率 8%，希望偵測到 2% 的提升，α=0.05，Power=0.80）
4. 說明如何避免多重比較問題（若同時監控多個指標）
5. 設計一個「實驗不純染」的防護措施（避免 A 組用戶看到 B 組內容）

評估標準： - H0 和 H1 方向是否正確（H0 是「沒有效果」） - 樣本數計算過程是否合理 - 是否意識到多重比較風險並有對應方案 - 實驗設計的嚴謹性

練習 C：解讀 A/B Test 結果（10 分鐘）

以下是一個已完成的 A/B Test 結果，請回答問題：

[redacted]

問題： 1. 主要指標（點擊率）的結論是什麼？效果量如何評估？ 2. 「跳出率顯著下降（p=0.03）」是否應該列入上線理由？為什麼？ 3. 你的最終建議是什麼？（上線 / 延長測試 / 維持現狀）

講師標準答案要點： - 點擊率：統計顯著（p=0.021<0.05），但效果量 h=0.033 屬於微小效果 - 跳出率的 p=0.03 是 8 個次要指標同時比較的結果，需要 Bonferroni 校正後 α=0.00625，p=0.03 不再顯著 - 建議：需評估「0.9% 的點擊率提升」對業務的實際價值，若業務價值明確則可上線

時間分配： - 練習 A：20 分鐘個人實作 - 練習 B：15 分鐘個人實作 - 練習 C：10 分鐘閱讀與回答 - 講師講解：15 分鐘標準答案與討論

模組總結

本模組的核心主線

統計不是數學課，是決策工具。
從「均值有差異」到「差異在統計上可靠」到「差異在業務上值得行動」——這三步缺一不可。

三個不能忘的原則

1. 均值不夠，看分佈：相同均值可以有完全不同的故事
2. $p < 0.05 \neq$ 差異很大：統計顯著性和實際顯著性是兩件事，需要效果量才完整
3. 多重比較是陷阱：同時監控 20 個指標，必然有一個「顯著」，要事前聲明主要指標

下一步：M7 ML 前導與學習路線

本模組建立的統計直覺（分佈、假設檢定、相關性）是機器學習的理論基礎。M7
將展示統計如何延伸成學習算法——從線性回歸的最小二乘法，到邏輯回歸的最大似然估計，到決策樹的信息增

延伸閱讀

參考文獻與引用來源

統計學經典文獻

1. Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences, 2nd Edition. Lawrence Erlbaum Associates. — 效果量（effect size）標準的原始來源。Cohen's d 門檻：0.2（小）、0.5（中）、0.8（大）。Cohen 本人曾警告這些是相對標準，應依研究領域調整。
2. Wasserstein, R.L. & Lazar, N.A. (2016). "The ASA Statement on Statistical Significance and p-Values." The American Statistician, 70(2), 129-133. — 美國統計學會（ASA）的官方聲明，釐清 p-value 的六項原則。doi.org/10.1080/00031305.2016.1154108
3. Nuzzo, R. (2014). "Statistical Errors." Nature, 506, 150-152. — 關於 p-value 誤用的重要短文。doi.org/10.1038/506150a

分布與定理

1. 68-95-99.7 法則 — 常態分布的經驗法則，源自 Gauss 的常態分布理論（1809）。任何統計學教科書均有說明，例如 Devore, J.L. Probability and Statistics for Engineering and the Sciences。
2. 中央極限定理（CLT） — 由 Laplace (1810) 首次嚴格陳述，Lindeberg (1922) 與 Feller (1935) 進一步推廣。現代教科書參考：Rice, J.A. (2006). Mathematical Statistics and Data Analysis, 3rd Edition.

相關性門檻

1. 相關係數強度分類（|r| > 0.7 強、0.3-0.7 中、< 0.3 弱） — 此為 Cohen (1988) 提出的慣例門檻。需注意：不同領域的「強相關」標準不同（社會科學 r=0.3 已算中度，物理學 r=0.9 才算起步）。

多重比較問題

1. 多重檢定問題 — Bonferroni 校正原始來源：Bonferroni, C.E. (1936). "Teoria statistica delle classi e calcolo delle probabilita." 現代說明可參見 Bland, J.M. & Altman, D.G. (1995). "Multiple

significance tests: the Bonferroni method." BMJ, 310, 170.

A/B Test 實務

1. Kohavi, R., Tang, D., & Xu, Y. (2020). Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing. Cambridge University Press. — Amazon/Microsoft 的 A/B Test 工業標準參考書。

冰淇淋與溺水相關性

1. 冰淇淋銷量 vs 溺水事故 — 此為統計學教學中的經典虛假相關案例（confounding variable: 氣溫）。學術引用可參見 Vigen, T. (2015). Spurious Correlations. Hachette Books. 以及 tylervigen.com/spurious-correlations

延伸閱讀

1. Downey, A.B. (2014). Think Stats, 2nd Edition. O'Reilly Media. — 用 Python 程式碼建立統計直覺。免費版本：greenteapress.com/thinkstats2
2. Gelman, A. & Hill, J. (2006). Data Analysis Using Regression and Multilevel/Hierarchical Models. Cambridge University Press. — 進階統計建模的標準參考書
3. scipy.stats 官方文件 — docs.scipy.org/doc/scipy/reference/stats.html